

Employee Manager — kompleksowe zadanie testowe

Aplikacja bazowa: Simple Test API (FastAPI + Vanilla JS)

Niniejszy dokument opisuje zadanie projektu końcowego dla uczestników kursu QA. Zadanie bazuje na aplikacji „Employee Manager” (repozytorium „Simple Test API”) i obejmuje pełny cykl pracy testera: od analizy wymagań, przez projektowanie i dokumentowanie testów, po automatyzację testów API oraz UI.s

Spis treści

1. Cel projektu	2
2. Opis testowanej aplikacji	2
3. Zakres zadania — etapy realizacji	3
Etap 1. Przegląd wymagań i pytania doprecyzowujące	3
Etap 2. Projektowanie przypadków testowych (techniki czarnoskrzynkowe, wyłącznie UI)	3
Etap 3. Podstawowy plan testów	4
Etap 4. Test case'y w TestRail	4
Etap 5. Zgłaszanie defektów w JIRA	4
Etap 6. Uruchomienie kodu w Pythonie	4
Etap 7. Wysłanie kodu Python do GitHub	4
Etap 8. Kolekcja Postman	4
Etap 9. Testy API (Python + pytest)	5
Etap 10. Testy UI (Python + pytest + Selenium)	5
4. Wymagane narzędzia i technologie	6
5. Co należy oddać (deliverables)	6
6. Harmonogram	6
7. Przydatne linki	6

1. Cel projektu

Celem projektu jest przeprowadzenie kompletnego procesu testowania wybranej aplikacji webowej — od analizy dokumentacji wymagań, przez ręczne projektowanie i wykonanie testów, zarządzanie defektami, aż po automatyzację testów na poziomie API i UI. Zadanie ma odwzorowywać realną pracę testera oprogramowania w zespole projektowym.

Po zakończeniu projektu uczestnik powinien potrafić:

- przeanalizować dokumentację wymagań i wskazać w niej luki, niejasności oraz błędy,
- zaprojektować przypadki testowe z użyciem technik czarnoskrzynkowych,
- opracować podstawowy plan testów,
- zarządzać przypadkami testowymi w narzędziu klasy Test Case Management (TestRail),
- zgłaszać i dokumentować defekty w narzędziu do zarządzania zadaniami (JIRA),
- uruchomić i skonfigurować środowisko Python dla projektu testowego,
- pracować z systemem kontroli wersji Git/GitHub,
- zbudować kolekcję testów API w Postman,
- zautomatyzować testy API przy użyciu Python + pytest,
- zautomatyzować testy UI przy użyciu Python + pytest + Selenium.

2. Opis testowanej aplikacji

„Employee Manager” to aplikacja klient-serwer do zarządzania pracownikami (dodawanie, edycja, usuwanie, przeglądanie listy, oznaczanie urlopu).

- Backend: FastAPI (Python), walidacja danych przez Pydantic, dane przechowywane w pamięci.
- Frontend: HTML + CSS + Vanilla JavaScript, komunikacja z backendem przez Fetch API.
- Aplikacja uruchamiana lokalnie pod adresem: <http://127.0.0.1:8000/>
- Dokumentacja API (Swagger): <http://127.0.0.1:8000/docs>
- Pełna specyfikacja wymagań: plik FUNCTIONAL_REQUIREMENTS.md w repozytorium projektu.

Model pracownika obejmuje pola: name (string), salary (integer), age (integer), position (enum: Junior QA / Mid QA / Senior QA / QA Lead) oraz on_leave (boolean). Pole id jest generowane automatycznie przez backend.

Metoda	Endpoint	Opis
GET	/health	Healthcheck API
GET	/api/employees	Pobranie listy pracowników
POST	/api/employees	Dodanie pracownika
PUT	/api/employees/{id}	Aktualizacja pracownika
DELETE	/api/employees/{id}	Usunięcie pracownika
POST	/api/employees/reset	Usunięcie wszystkich pracowników i reset licznika ID

3. Zakres zadania — etapy realizacji

Projekt składa się z dziesięciu powiązanych ze sobą etapów. Zaleca się realizację w podanej kolejności, ponieważ wyniki wcześniejszych etapów (przypadki testowe, defekty) są wykorzystywane w kolejnych.

Etap 1. Przegląd wymagań i pytania doprecyzowujące

Przeanalizuj dokument FUNCTIONAL_REQUIREMENTS.md w całości.

- Zidentyfikuj minimum 5 błędów, niejasności, sprzeczności lub braków w wymaganiach (np. brakujące wartości graniczne, niejednoznaczne sformułowania, sprzeczności między sekcjami, brak specyfikacji zachowania w przypadkach brzegowych).
- Dla każdego zidentyfikowanego problemu sformułuj konkretne, możliwe do zadania pytanie doprecyzowujące do Product Ownera / zespołu deweloperskiego.
- Wynik przedstaw w formie tabeli: Lp. | Sekcja wymagań | Opis problemu | Pytanie doprecyzowujące.

Uwaga: celem tego etapu nie jest naprawa wymagań, lecz wykazanie umiejętności krytycznej analizy dokumentacji — typowej pracy testera na etapie przeglądu wymagań (requirements review).

Etap 2. Projektowanie przypadków testowych (techniki czarnoskrzynkowe, wyłącznie UI)

Zaprojektuj przypadki testowe walidujące pola formularza pracownika: Name, Salary, Age, Position, On vacation.

- Zastosuj techniki czarnoskrzynkowe: analizę klas równoważności (Equivalence Partitioning) oraz analizę wartości brzegowych (Boundary Value Analysis). Tam, gdzie to zasadne, zastosuj również tabelę decyzyjną (Decision Table) dla kombinacji pól.
- Testowanie odbywa się WYŁĄCZNIE przez interfejs użytkownika (formularz w przeglądarce) — bez wysyłania zapytań bezpośrednio do API (np. przez Postman czy curl).
- Minimalna liczba przypadków: 6 dla Name, 6 dla Salary, 6 dla Age, 4 dla Position, 2 dla On vacation — uwzględniając przypadki pozytywne i negatywne.
- Każdy przypadek testowy musi zawierać: ID, tytuł, warunki wstępne (precondition), kroki testowe, dane wejściowe, oczekiwany rezultat.

Pole	Ograniczenia
Name	wymagane; długość 1–50 znaków; litery, cyfry, pojedyncze spacje; bez znaków specjalnych
Salary	typ integer; zakres 1–200000
Age	typ integer; zakres 18–65
Position	wymagane; wybór wyłącznie z listy: Junior QA, Mid QA, Senior QA, QA Lead
On leave	typ boolean (checkbox); domyślnie odznaczony (false)

Punkt odniesienia dla wartości granicznych (na podstawie FUNCTIONAL_REQUIREMENTS.md, sekcja 4):

Etap 3. Podstawowy plan testów

Opracuj plan testów (Test Plan) w formie dokumentu (Word lub PDF), zawierający co najmniej następujące elementy:

1. Wprowadzenie i cel testowania
2. Zakres testów (co jest objęte testami, a co świadomie pominięte)
3. Strategia testowania (rodzaje testów: funkcjonalne, walidacja pól, UI, regresja)
4. Środowisko testowe (system operacyjny, przeglądarka, wersja Pythona)
5. Role i odpowiedzialności
6. Kryteria wejścia i wyjścia (entry/exit criteria)
7. Kryteria akceptacji
8. Ryzyka projektowe i sposoby ich ograniczenia
9. Harmonogram realizacji

Etap 4. Test case'y w TestRail

- Załóż projekt (może być darmowa wersja trial) w narzędziu TestRail pod nazwą „Employee Manager”.
- Przenieś przypadki testowe zaprojektowane w Etapie 2 do TestRail, grupując je w sekcje odpowiadające polom formularza (Name, Salary, Age, Position, On vacation).
- Uzupełnij pola takie jak: tytuł, kroki, oczekiwany rezultat, priorytet, typ testu.
- Wykonaj zrzuty ekranu potwierdzające utworzenie projektu i przypadków testowych.

Etap 5. Zgłaszanie defektów w JIRA

- Załóż projekt (darmowy plan) w JIRA.
- Wykonaj testy manualne zgodnie z przypadkami testowymi z TestRail.
- Dla każdego znalezionej błędzie zgłoś defekt zawierający: tytuł, kroki reprodukcji, wynik oczekiwany, wynik rzeczywisty, środowisko, priorytet/severity, zrzut ekranu lub inny dowód.
- Zgłoś minimum 3 defekty. Jeśli podczas testów funkcjonalnych nie uda się znaleźć wystarczającej liczby błędów, dopuszczalne jest zgłoszenie niezgodności z wymaganiami UX/UI.

Etap 6. Uruchomienie kodu w Pythonie

- Uruchom aplikację lokalnie (python run.py) i potwierdź jej działanie pod adresem <http://127.0.0.1:8000/>.
- Instrukcje uruchomienia aplikacji znajdziesz w repozytorium
- Załącz zrzut ekranu terminala z uruchomioną aplikacją oraz zrzut działającej aplikacji w przeglądarce.

Etap 7. Wysyłanie kodu Python do GitHub

- Załóż własne repozytorium na GitHub (może być prywatne — z dostępem udostępnionym prowadzącemu).
- Wypchnij (push) do repozytorium cały kod wytworzony w ramach zadania: testy API, testy UI, ewentualne skrypty pomocnicze oraz kolekcję Postman.
- Repozytorium powinno zawierać czytelny plik README.md z instrukcją instalacji i uruchomienia testów.

Etap 8. Kolekcja Postman

- Zbuduj kolekcję Postman obejmującą wszystkie endpointy API projektu (patrz tabela w sekcji 2 niniejszego dokumentu) lub wymagania funkcjonalne.
- Dla każdego żądania skonfiguruj przykładowe dane wejściowe oraz podstawowe testy w zakładce Tests (m.in. weryfikacja kodu statusu, struktury odpowiedzi).
- Uwzględnij scenariusze błędów: 422 (niepoprawne dane), 404 (nieistniejący id).
- Wyeksportuj kolekcję do pliku .json i dołącz go do repozytorium GitHub.

Etap 9. Testy API (Python + pytest)

- Zaimplementuj automatyczne testy API z użyciem pytest oraz biblioteki requests
- Pokryj testami: happy path dla wszystkich endpointów, walidację pól (kod 422), obsługę nieistniejącego zasobu (kod 404) oraz działanie endpointu resetu (POST /api/employees/reset).
- Testy mogą być niezależne od kolejności wykonania — np. przez zastosowanie fixture resetującej dane przed każdym testem.
- Wygeneruj i dołącz raport z wykonania testów (np. pytest --html lub log z konsoli).

Etap 10. Testy UI (Python + pytest + Selenium)

- Zaimplementuj automatyczne testy UI z użyciem pytest oraz Selenium WebDriver.
 - Pokryj co najmniej: dodanie pracownika, edycję pracownika, usunięcie pracownika, walidację pól formularza (komunikaty błędów), działanie modala potwierdzenia resetu danych, przełączanie motywu dark/light.
 - Stosuj jawne oczekiwania (explicit waits) — unikaj time.sleep().
 - Zalecana struktura projektu zgodna z wzorcem Page Object Model (POM).
-

4. Wymagane narzędzia i technologie

Obszar	Narzędzie / technologia
Środowisko uruchomieniowe	Python 3.13+, venv
Kontrola wersji	Git, GitHub
Zarządzanie przypadkami testowymi	TestRail
Zarządzanie defektami	JIRA
Testy API — narzędzie manualne	Postman
Automatyzacja API	pytest, requests / http
Automatyzacja UI	pytest, Selenium WebDriver
Dokumentacja	Word / PDF (plan testów, przegląd wymagań)

5. Co należy oddać (deliverables)

- Dokument z przeglądem wymagań i listą pytań (Etap 1).
- Zestaw przypadków testowych z uzasadnieniem zastosowanych technik (Etap 2).
- Plan testów w formacie Word/PDF (Etap 3).
- Link do projektu w TestRail, eksport przypadków testowych, lub zrzuty ekranu (Etap 4).
- Link do projektu w JIRA, eksport zgłoszonych defektów, lub zrzuty ekranu (Etap 5).
- Zrzuty ekranu potwierdzające lokalne uruchomienie aplikacji (Etap 6).
- Link do repozytorium GitHub z kodem projektu (Etap 7, 9, 10).
- Plik kolekcji Postman (.json) (Etap 8).
- Kod testów API wraz z raportem z wykonania (Etap 9).
- Kod testów UI zgodny z dobrymi praktykami (Etap 10).

6. Harmonogram

Termin oddania projektu: Tydzień po zakończeniu szkolenia

7. Przydatne linki

- Repozytorium projektu: https://github.com/pwalaszkowski/codebrainers_simple_test_api
- Dokumentacja wymagań: FUNCTIONAL_REQUIREMENTS.md (w repozytorium)
- Dokumentacja API (Swagger): <http://127.0.0.1:8000/docs> (po lokalnym uruchomieniu aplikacji)
- TestRail: <https://www.testrail.com/>
- JIRA: <https://www.atlassian.com/software/jira>
- Postman: <https://www.postman.com/>
- pytest: <https://docs.pytest.org/>
- Selenium: <https://www.selenium.dev/documentation/>

8. Miejsce na rozwiązania

Rozwiązane zadania, tabelki, zrzuty ekranów lub linki do Github' umieść poniżej

Rozwiązanie Etap 1

Lp.	Sekcja wymagań	Opis problemu	Pytanie do Product Ownera
1	12.4 / 12.5 Autoryzacja	Nie jest napisane od kiedy liczy się 10 minut ważności tokena.	Od kiedy liczymy te 10 minut? Od momentu zalogowania czy od ostatniej aktywności użytkownika?
2	3 / 6 / 9 Nazewnictwo pola urlopu	W dokumentacji występują różne nazwy tego samego pola: On leave, On vacation i Vacation.	Która nazwa jest poprawna i powinna być używana w aplikacji?
3	4.1 Walidacja pola Name (Spacja)	Nie jest opisane, jak aplikacja ma zachować się, gdy użytkownik wpisze spację na początku lub na końcu imienia i nazwiska.	Czy takie spacje powinny być automatycznie usuwane, czy użytkownik powinien dostać komunikat o błędzie?
4	4.1 Walidacja pola Name (Cyfry)	Opis mówi o imieniu i nazwisku, ale regex pozwala wpisywać cyfry.	Czy pole Name powinno pozwalać na wpisywanie cyfr, czy tylko liter i spacji?
5	4.1 Walidacja pola Name (Limit 50 znaków)	W wymaganiach jest informacja, że pole może mieć maksymalnie 50 znaków, ale nie jest napisane, czy do limitu wliczają się również spacje.	Czy do limitu 50 znaków wliczają się również spacje?
6	10.4 / 13.5 Obsługa błędów	Nie opisano jak mają wyglądać komunikaty, gdy jest kilka błędów jednocześnie.	Czy wszystkie błędy mają być wyświetlane w jednym Error Boxie, czy osobno przy polach?
7	13.1 Wygląd na różnych ekranach	Napisano tylko, że aplikacja ma dobrze działać na mniejszych ekranach, ale brakuje konkretów o telefonach czy tabletach.	Na jakich urządzeniach powinna być sprawdzana responsywność aplikacji? Czy wystarczy telefon, czy również tablet?
8	12.5 Wygaśnięcie sesji	Nie jest opisane, czy po wygaśnięciu sesji użytkownik powinien dostać informację, dlaczego został przeniesiony na stronę logowania.	Czy po wygaśnięciu sesji aplikacja powinna wyświetlić komunikat informujący o konieczności ponownego zalogowania?

Rozwiązanie Etap 2:

Środowisko testowe

- **System:** Ubuntu 24.04.4 LTS
- **Kernel:** WSL2 — 6.18.33.2-microsoft-standard-WSL2
- **Architektura:** x86_64
- **Przeglądarka:** Google Chrome for Linux
- **Wersja Chrome:** 151.0.7922.71 (Official Build) (x86_64)
- **Nazwa aplikacji:** Employee Manager
- **Wersja aplikacji:** 1.3.0
- **Adres:** <http://127.0.0.1:8000/>

Warunki wspólne dla wszystkich przypadków:

Użytkownik jest zalogowany i znajduje się na stronie głównej aplikacji. Formularz Add Employee jest dostępny.

Domyślne dane testowe

Pole	Wartość
Name	Jan Kowalski

Salary	3000
Age	30
Position	Junior QA
On vacation	odznaczone

NAME

Id.	Tytuł	Warunki wstępne	Kroki testowe	Dane wejściowe	Oczekiwany rezultat	Status
TC-NAME-001	Poprawna wartość w polu Name (EP)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Domyślne	Pracownik zostaje dodany.	PASS
TC-NAME-002	Minimalna długość pola Name (BVA)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Name: A Pozostałe pola: domyślne	Pracownik zostaje dodany.	PASS
TC-NAME-003	Maksymalna długość pola Name (BVA)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Name: Jan Kowalski Adam Nowak Piotr Zielinski Tomasz Now Pozostałe pola: domyślne	Pracownik zostaje dodany.	PASS
TC-NAME-004	Puste pole Name (BVA)	☒	1. Pozostaw pole Name puste. 2. Pozostałe pola pozostaw z domyślnymi danymi. 3.Kliknij Add.	Name: puste (0 znaków) Pozostałe pola: domyślne	Pracownik nie zostaje dodany do tabeli. Wyświetla się komunikat o błędzie walidacji pola Name.	PASS
TC-NAME-005	Przekroczenie maksymalnej długości pola Name (BVA)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Name: Jan Kowalski Adam Nowak Piotr Zielinski Tomasz Nowa Pozostałe pola: domyślne	Pole Name nie pozwala na wprowadzenie więcej niż 50 znaków. Nadmiarowy znak zostaje pominięty. Pracownik zostaje dodany z wartością Name zawierającą maksymalnie 50 znaków.	PASS
TC-NAME-006	Wpisanie cyfr w polu Name (EP)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Name: Jan 123 Kowalski Pozostałe pola: domyślne	Pracownik zostaje dodany.	PASS
TC-NAME-007	Wpisanie znaków specjalnych w polu Name (EP)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Name: Jan@Kowalski Pozostałe pola: domyślne	Pracownik nie zostaje dodany do tabeli. Wyświetla się komunikat o błędzie walidacji pola Name.	PASS
TC-NAME-008	Spacje na początku i końcu pola Name (EP)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Name: " Jan Kowalski " Pozostałe pola: domyślne	Przypadek oczekuje na odpowiedź PO	BLOCK

SALARY

Id.	Tytuł	Warunki wstępne	Kroki testowe	Dane wejściowe	Oczekiwany rezultat	Status
TC-SALARY-001	Minimalna wartość pola Salary (BVA)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Salary: 1 Pozostałe pola: domyślne	Pracownik zostaje dodany.	PASS
TC-SALARY-002	Maksymalna wartość pola Salary (BVA)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Salary: 200000 Pozostałe pola: domyślne	Pracownik zostaje dodany.	PASS
TC-SALARY-003	Wartość poniżej minimalnej wartości Salary (BVA)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Salary: 0 Pozostałe pola: domyślne	Pracownik nie zostaje dodany do tabeli. Wyświetla się komunikat o błędzie walidacji pola Salary.	PASS
TC-SALARY-004	Wartość powyżej maksymalnej wartości Salary (BVA)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Salary: 200001 Pozostałe pola: domyślne	Pracownik nie zostaje dodany do tabeli. Wyświetla się komunikat o błędzie walidacji pola Salary.	PASS
TC-SALARY-005	Poprawna wartość pola Salary (EP)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Domyślne	Pracownik zostaje dodany.	PASS
TC-SALARY-006	Wartość ujemna w polu Salary (EP)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Salary: -100 Pozostałe pola: domyślne	Pracownik nie zostaje dodany do tabeli. Wyświetla się komunikat o błędzie walidacji pola Salary.	PASS
TC-SALARY-007	Wartość dziesiętna w polu Salary (EP)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Salary: 3000.50 Pozostałe pola: domyślne	Pracownik nie zostaje dodany do tabeli. Wyświetla się komunikat	PASS

					o błędzie walidacji pola Salary.	
--	--	--	--	--	----------------------------------	--

AGE

Id.	Tytuł	Warunki wstępne	Kroki testowe	Dane wejściowe	Oczekiwany rezultat	Status
TC-AGE-001	Minimalna wartość pola Age (BVA)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Age: 18 Pozostałe pola: domyślne	Pracownik zostaje dodany.	PASS
TC-AGE-002	Maksymalna wartość pola Age (BVA)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Age: 65 Pozostałe pola: domyślne	Pracownik zostaje dodany.	PASS
TC-AGE-003	Wartość poniżej minimalnej wartości Age (BVA)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Age: 17 Pozostałe pola: domyślne	Pracownik nie zostaje dodany do tabeli. Wyświetla się komunikat o błędzie walidacji pola Age.	PASS
TC-AGE-004	Wartość powyżej maksymalnej wartości Age (BVA)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Age: 66 Pozostałe pola: domyślne	Pracownik nie zostaje dodany do tabeli. Wyświetla się komunikat o błędzie walidacji pola Age.	PASS
TC-AGE-005	Poprawna wartość pola Age (EP)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Domyślne	Pracownik zostaje dodany.	PASS
TC-AGE-006	Wartość ujemna w polu Age (EP)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Age: -20 Pozostałe pola: domyślne	Pracownik nie zostaje dodany do tabeli. Wyświetla się komunikat o błędzie walidacji pola Age.	PASS
TC-AGE-007	Wartość dziesiętna w polu Age (EP)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Age: 30.50 Pozostałe pola: domyślne	Pracownik nie zostaje dodany do tabeli. Wyświetla się komunikat o błędzie walidacji pola Age.	PASS

POSITION

Id.	Tytuł	Warunki wstępne	Kroki testowe	Dane wejściowe	Oczekiwany rezultat	Status
TC-POSITION-001	Brak wybranej pozycji (EP)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Position: Select position Pozostałe pola: domyślne	Pracownik nie zostaje dodany do tabeli. Wyświetla się komunikat o błędzie walidacji pola Position	PASS
TC-POSITION-002	Wybór Junior QA (EP)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Position: Junior QA Pozostałe pola: domyślne	Pracownik zostaje dodany.	PASS
TC-POSITION-003	Wybór Mid QA (EP)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Position: Mid QA Pozostałe pola: domyślne	Pracownik zostaje dodany.	PASS
TC-POSITION-004	Wybór Senior QA (EP)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Position: Senior QA Pozostałe pola: domyślne	Pracownik zostaje dodany.	PASS
TC-POSITION-005	Wybór QA Lead (EP)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Kliknij Add	Position: QA Lead Pozostałe pola: domyślne	Pracownik zostaje dodany.	PASS

ON LEAVE

Id.	Tytuł	Warunki wstępne	Kroki testowe	Dane wejściowe	Oczekiwany rezultat	Status
TC-ONLEAVE-001	Pole On leave odznaczone (EP)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Pozostaw checkbox On leave odznaczony 3. Kliknij Add	On leave: odznaczone Pozostałe pola: domyślne	Pracownik zostaje dodany a pole On leave ma stan odznaczony.	PASS
TC-ONLEAVE-002	Pole On leave zaznaczone (EP)	☒	1. Wprowadź dane testowe zgodnie z przypadkiem. 2. Zaznacz checkbox On leave 3. Kliknij Add	On leave: zaznaczone Pozostałe pola: domyślne	Pracownik zostaje dodany a pole On leave ma stan zaznaczony.	PASS

Tabela decyzyjna – Add Employee

Warunek / Reguła	R1	R2	R3	R4	R5
Name poprawny	✓	✗	✓	✓	✓
Salary poprawny	✓	✓	✗	✓	✓

Age poprawny	✓	✓	✓	X	✓
Position poprawny	✓	✓	✓	✓	X
Oczekiwany rezultat	Dodanie	Brak dodania	Brak dodania	Brak dodania	Brak dodania

Rozwiązanie Etap 3:

Plan testów aplikacji Employee Manager został przygotowany w osobnym dokumencie, Employee Manager – Test Plan w repozytorium GitHub w katalogu docs/.

Rozwiązanie Etap 4:

Przypadki testowe zostały utworzone i wykonane w TestRail.

Dokumentacja i dowody znajdują się w repozytorium GitHub w katalogu testrail/.

Rozwiązanie Etap 5:

Podczas testów manualnych zidentyfikowano i zgłoszono **3 defekty w JIRA**:

- **KAN-1** – Previous validation error remains visible after entering Edit mode
- **KAN-2** – Deleting an employee in edit mode blocks further application use
- **KAN-3** – Previously issued authentication token remains valid after logout

Dokumentacja zgłoszeń oraz informacje o dowodach znajdują się w repozytorium GitHub w katalogu jira/.

Rozwiązanie Etap 6:

Aplikacja Employee Manager została uruchomiona lokalnie i zweryfikowana w przeglądarce.

Do projektu dołączono zrzuty ekranu potwierdzające:

- uruchomienie aplikacji w terminalu,
- działanie aplikacji w przeglądarce.

Dowody znajdują się w docs/screenshots/.

Rozwiązanie Etap 7:

Projekt został umieszczony w repozytorium GitHub.

Repozytorium zawiera dokumentację QA, przypadki testowe, kolekcję Postman, testy automatyczne API i UI oraz raporty z wykonania.

GitHub: <https://github.com/mstobrawa/employee-manager-qa>

Rozwiązanie Etap 8:

Plik kolekcji znajduje się w postman/employee-manager-qa-api.postman_collection.json.

Rozwiązanie Etap 9:

Testy znajdują się w repozytorium GitHub w folderze:

tests/api/

Raport z wykonania znajduje się w:

docs/pytest-report.html

Rozwiązanie Etap 10:

Testy znajdują się w repozytorium GitHub w folderze:

tests/ui/

Raport z wykonania znajduje się w:

docs/selenium-report.html